

SUBMITTED BY:

ALI HAMZA

SUBMITTED TO:

MAM MUNEEBA KHAN

REGISTRATION NO:

UW-24-CY-BS-027

LAB TITLE:

SECURE SOFTWARE DESIGN AND DEVELOPMENT

LAB #03

Contents

1. ***Aim and Objectives*** 3
2. ***Scenario and DFD with Trust Boundaries*** 3
3. ***Asset Table*** 5
4. ***Threat List and STRIDE Summary*** 5
5. ***Risk Register and Risk Matrix Summary*** 8
6. ***SDLC Mapping and Analysis*** 10
7. ***GitHub Work*** 11
8. ***Answers to Practice Questions*** 12
9. ***Conclusion*** 14
10. ***Role Sign-Off*** 14

1. Aim and Objectives

Aim: To identify and analyze assets, threats, vulnerabilities, and risks within a software system, map security activities to SDLC phases, and publish the threat model using GitHub.

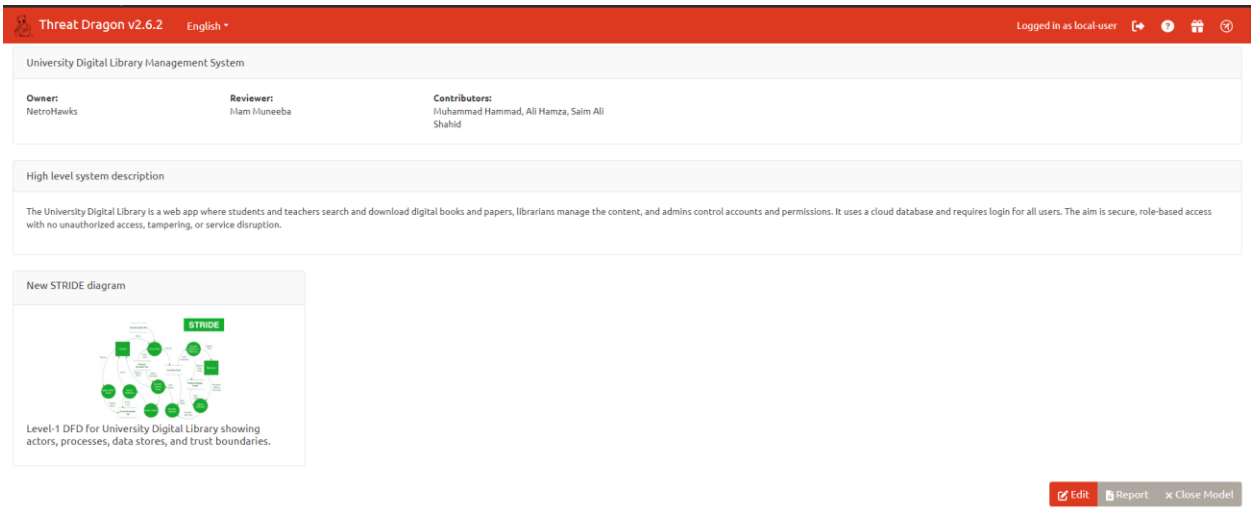
Objectives:

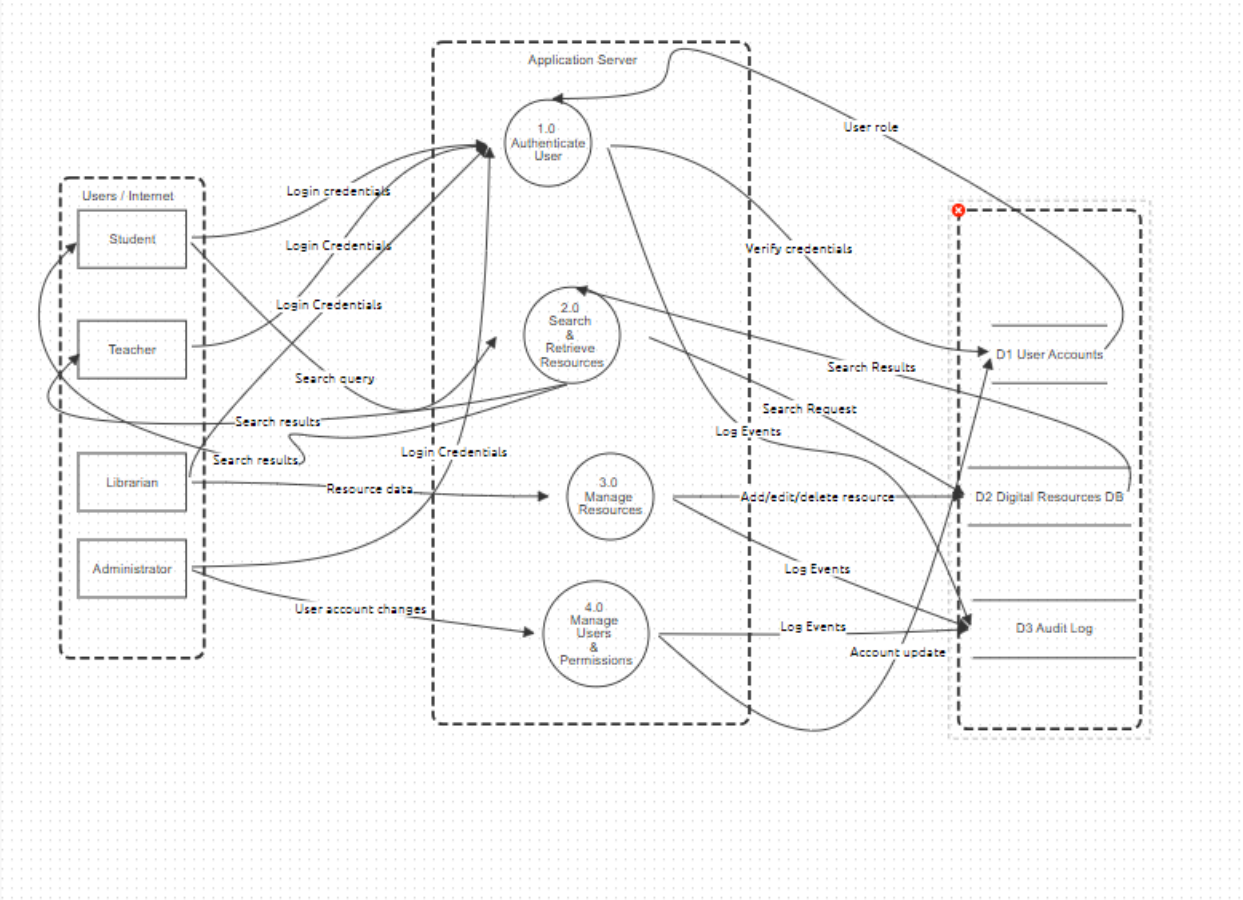
1. Identify assets and classify them by security objectives (CIA).
2. Mark trust boundaries and entry points on a DFD.
3. Find threats using the STRIDE method and record them in OWASP Threat Dragon.
4. Link threats to vulnerabilities, rate risks, and propose mitigations.
5. Relate security activities to the correct SDLC phases.
6. Use GitHub to store the model and track threats as issues.

2. Scenario and DFD with Trust Boundaries

Scenario: Our system is a University Digital Library Management System. It is a web application where students and teachers search and download digital books/papers. Librarians manage the content, and administrators control accounts and permissions. The system uses a cloud database and requires login for all users. The goal is to ensure secure, role-based access with no unauthorized access, tampering, or service disruption.

DFD and Trust Boundaries:





- **TB1 (Users / Internet):** Between external actors (Student, Teacher, Admin, Librarian) and the Application Server.
- **TB2 (Application Server):** Contains the internal processes (1.0 to 4.0).
- **TB3 (Cloud / Data Zone):** Contains the Data Stores (D1, D2, D3).

3. Asset Table

ID	Asset	Where it lives (DFD element)	Objective (C / I / A)	Value
A1	User Passwords & Roles	D1 User Accounts	C, I	High
A2	Digital Resource Files (Books/Papers)	D2 Digital Resources DB	C, I	High
A3	Resource Metadata	D2 Digital Resources DB	I	Medium
A4	Audit Log Records	D3 Audit Log	I	High
A5	Session Tokens	Flow (1.0 to Users)	C, I	High
A6	Search Results Data	Flow (2.0 to Users)	C	Medium
A7	System Uptime / Availability	Process 2.0, App Server	A	High
A8	Librarian Upload Data	Flow (Librarian to 3.0)	C, I	Medium
A9	Admin Account Credentials	Flow (Admin to 1.0)	C, I	High
A10	User Account Update Flow	Flow (Admin to 4.0)	I	High

4. Threat List and STRIDE Summary

- **Total Threats:** 12
- **STRIDE Counts:**
  - **S**poofing: 2 *(Threats #1 and #7)*
  - **T**ampering: 4 *(Threats #2, #8, #13, #14)*
  - **R**epudiation: 1 *(Threat #3)*
  - **I**nformation Disclosure: 2 *(Threats #4 and #9)*
  - **D**enial of Service: 2 *(Threats #5 and #10)*
  - **E**levation of Privilege: 1 *(Threat #6)*

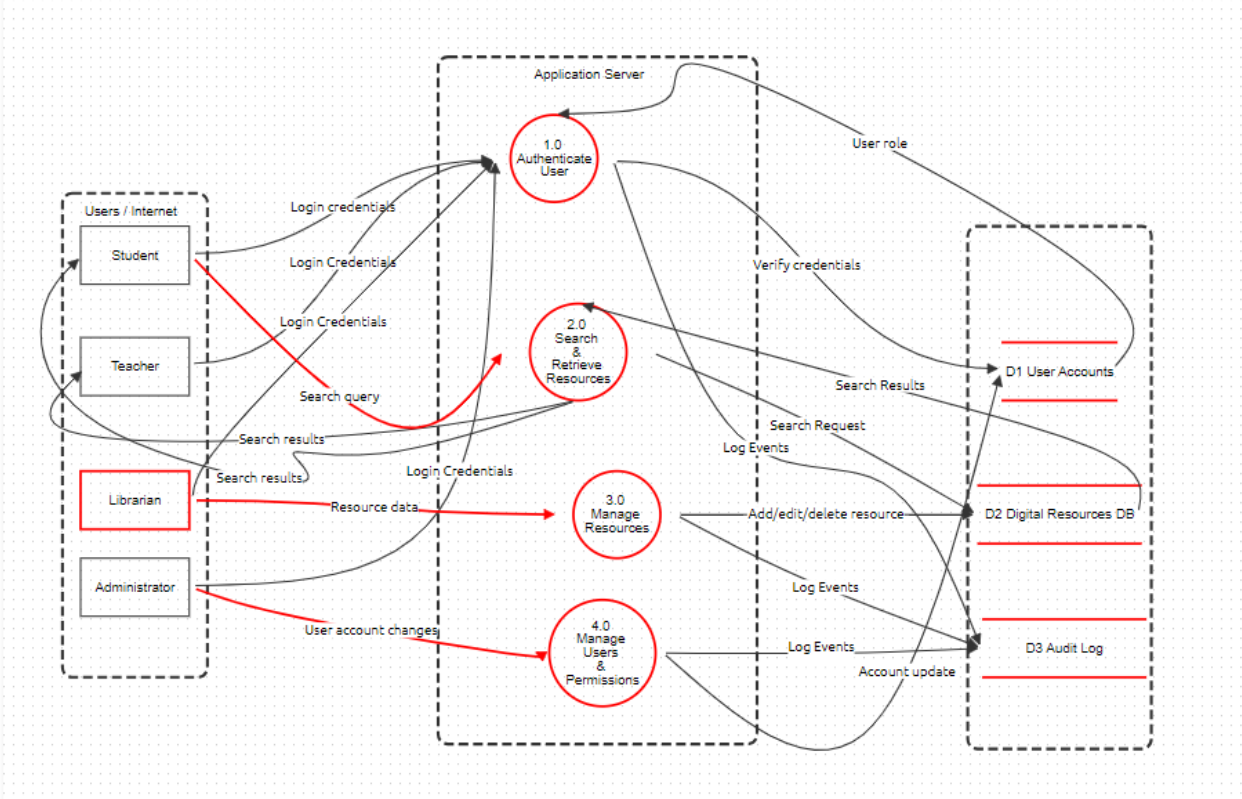
Executive Summary

High level system description

The University Digital Library is a web app where students and teachers search and download digital books and papers, librarians manage the content, and admins control accounts and permissions. It uses a cloud database and requires login for all users. The aim is secure, role-based access with no unauthorized access, tampering, or service disruption.

Summary

Metric	Total
Total Threats	12
Total Mitigated	0
Total Open	12
Open / Critical Severity	0
Open / High Severity	8
Open / Medium Severity	3
Open / Low Severity	1



Librarian (Actor)

Description: Adds, edits, and removes books/papers. Write access to resource database.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
7	Stolen session token used to upload fake resources.	Spoofing	High	Open	12	An attacker steals a Librarian's active session cookie via cross-site scripting (XSS). They use the stolen session to upload malicious PDF files to the library without needing the Librarian's password.	Implement short session timeouts, use HttpOnly and Secure flags on cookies, and enforce re-authentication for upload actions.

1.0 Authenticate User (Process)

Description: Verifies username/password against User Accounts DB and assigns role. Handles credentials

Number	Title	Type	Severity	Status	Score	Description	Mitigations
1	Attacker guesses a Teacher's password	Spoofing	High	Open	20	An external attacker uses an automated brute-force tool against the login process. Because there is no limit on failed attempts, they eventually guess a teacher's password and gain unauthorized access to download restricted academic papers.	Lock the account after 5 failed attempts, enforce rate limiting, and require 12-character passwords with MFA for teachers.

2.0 Search & Retrieve Resources (Process)

Description: Handles search queries and returns matching books/papers from resources DB.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
5	Botnet floods the search function	Denial of service	High	Open	16	An attacker uses a botnet to send thousands of automated search queries per second to Process 2.0. The application server becomes overloaded and crashes, preventing legitimate students from searching for resources.	Implement a Web Application Firewall (WAF) and API rate limiting to block high-volume traffic from single IP addresses.

3.0 Manage Resources (Process)

Description: Librarian adds, edits, or deletes books and papers in the resources DB.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
3	Librarian denies deleting a critical resource	Repudiation	Medium	Open	8	A librarian deletes an important digital resource from the library. Later, they claim they never did it. Because the system lacks a proper audit trail linking actions to specific user IDs, the university cannot prove who performed the deletion.	Ensure Process 3.0 sends a "Log Event" to D3 Audit Log containing the user ID, timestamp, and action performed.

4.0 Manage Users & Permissions (Process)

Description: Admin creates, updates, deletes user accounts and changes roles. Handles credentials Handles PII

Number	Title	Type	Severity	Status	Score	Description	Mitigations
6	Student manipulates account changes to gain Admin rights	Elevation of privilege	High	Open	20	A student intercepts the "User account changes" data flow and modifies the request to change their own role from 'Student' to 'Administrator'. The server processes the request without checking if the requester has the authority to make that change.	Implement strict server-side Role-Based Access Control (RBAC). Verify that the user making the request is an Admin before applying account updates.

D1 User Accounts (Store)

Description: Stores usernames, hashed passwords, roles, and personal details.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
8	Direct database modification of user roles	Tampering	Medium	Open	8	An attacker gains direct access to the D1 User Accounts database (e.g., via a misconfigured cloud port) and manually updates their account row to change their role to Administrator.	Restrict database access to only the application server's IP address, use strong database passwords, and encrypt the database at rest.
10	Login floods lock up the user database	Denial of service	Medium	Open	9	An attacker sends thousands of concurrent login requests to Process 1.0. This causes Process 1.0 to open too many connections to D1 User Accounts, causing the database to crash and preventing all users from logging in.	Implement database connection pooling limits and configure rate limiting on the authentication endpoint.

D2 Digital Resources DB (Store)

Description: Stores book/paper metadata, file paths, and resource records.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
4	Unencrypted database backup exposed	Information disclosure	High	Open	15	An attacker gains access to the cloud storage where the Digital Resources DB is backed up. Because the backup is stored in plain text, the attacker downloads all copyrighted materials and restricted research papers.	Encrypt the database at rest and enforce strict access controls on cloud storage buckets.

User account changes (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
13	Attacker alters account permissions in transit	Tampering	High	Open	15	An attacker intercepts the "User account changes" data flow using a Man-in-the-Middle attack. They modify the request payload to change a user's role from 'Student' to 'Administrator' before it reaches Process 4.0, granting them full control over the system.	Encrypt all internal data flows using TLS 1.3 to prevent interception, and implement server-side checksum validation to detect tampered payloads.

Search query (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
14	SQL injection in the search query (Critical)	Tampering	High	Open	25	A student enters malicious SQL code (e.g., ' OR 1=1 --) into the search query field. The application passes this input directly to D2 Digital Resources DB, allowing the attacker to dump the entire database.	Use parameterized queries (prepared statements) for all database interactions and implement strict input validation.

Resource data (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
2	Malicious file intercepted and altered in transit	Tampering	High	Open	16	A Man-in-the-Middle attacker intercepts the "Resource data" flow between the Librarian and Process 3.0. They alter the PDF file before it reaches the database, corrupting the integrity of the digital resource.	Enforce TLS 1.3 encryption for all data in transit and use file hashing to verify integrity upon receipt.

D3 Audit Log (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
9	Unauthorized reading of audit logs	Information disclosure	Low	Open	4	A curious student discovers that the D3 Audit Log is publicly accessible and reads the login history and IP addresses of other students and teachers.	Configure the audit log database with strict read-access controls so only the system administrator can view it.

5. Risk Register and Risk Matrix Summary

ID	Asset	Threat (STRIDE)	Vulnerability	L	I	Score	Level	Mitigation
R1	Teacher Accts	Spoofing: Password guess (1.0)	No failed login limit	4	5	20	Critical	Lockout, rate limit, MFA
R2	Resource Files	Tampering: File altered in transit	Unencrypted transmission	4	4	16	High	TLS 1.3, file hashing
R3	Audit Logs	Repudiation: Denied deletion (3.0)	No audit trail	2	4	8	Medium	Log events (User ID/time)
R4	Digital DB	Info Disc: Backup exposed (D2)	Unencrypted data at rest	3	5	15	High	Encrypt at rest, access controls
R5	App Server	DoS: Search flood (2.0)	No rate limiting	4	4	16	High	WAF, API rate limiting
R6	User Accts	EoP: Student gains Admin (4.0)	No server-side RBAC	4	5	20	Critical	Server-side RBAC checks
R7	Librarian	Spoofing: Stolen session token	Insecure cookies	3	4	12	High	Short timeouts, HttpOnly

R8	D1 User DB	Tampering: Direct DB role edit (D1)	Misconfigured cloud port	2	4	8	Medium	Restrict DB access, strong pwd
R9	D3 Audit Log	Info Disc: Unauthorized reading (D3)	Public logs	2	2	4	Low	Strict read-access controls
R10	D1 User DB	DoS: Login flood (D1)	No connection pooling	3	3	9	Medium	Connection pool limits
R11	Account Flow	Tampering: Altered permissions	Man-in-the-Middle	3	5	15	High	TLS 1.3, checksum validation
R12	Search Query	Tampering: SQL Injection	Unvalidated input	5	5	25	Critical	Parameterized queries

Risk Matrix Summary: We identified 3 Critical, 5 High, 3 Medium, and 1 Low risk. R12 (SQL Injection) is the most severe (Score: 25). All Critical and High risks require immediate mitigation before deployment.

6. SDLC Mapping and Analysis

Risk ID	Threat	Phase where it is prevented	Security Activity	Objective
R1	Password guessing on 1.0	Design, Implementation, Testing	Define lockout rule, code it, test it	Authenticity
R2	Malicious file altered in transit	Design, Deployment	Enforce TLS encryption between servers	Integrity
R3	Librarian denies deleting resource	Design, Implementation	Write audit log code to capture user ID and timestamp	Accountability
R5	Botnet floods the search function	Deployment, Maintenance	Configure WAF rules, set up rate limiting, monitor traffic	Availability
R6	Student gains Admin rights	Requirements, Testing	Define RBAC access-control rules, test that students cannot access admin panels	Authorisation
R9	Unauthorized reading of logs	Deployment, Maintenance	Apply strict read-access controls to the database	Confidentiality
R11	Attacker alters account permissions	Design, Implementation	Implement checksum validation on data flows	Integrity
R12	SQL injection in search query	Implementation, Testing	Use parameterized queries, run static analysis and penetration tests	Integrity

Short Analysis (Part C3):

1. **Most activities:** The *Design and Implementation* phases contain the most security activities because that is where the actual fixes (like encryption and input validation) are built into the code.
2. **Most threatened objective:** *Integrity* is the most threatened objective, as attackers repeatedly attempt to alter resources, permissions, and search queries.
3. **Expensive late fix:** R12 (SQL Injection) would be the most expensive to fix after release. A database breach would require notifying all users, restoring backups, and rebuilding the system's reputation.

7. GitHub Work

Repository URL: https://github.com/uw-24-cy-bs-051-byte/secure-sdd-lab02-group7

Group Contributions (Commit History):
The repository was built collaboratively, with all members contributing directly to the version control history, file uploads, and issue tracking.

- **M Hammad (051) – Group Lead & GitHub Maintainer:** Initialized the repository, uploaded the core Threat Dragon .json model and the generated PDF threat report, created issues for R2, R5, and R10, and finalized the README documentation.
- **Ali Hamza (027) – Asset and Boundary Analyst:** Reviewed the DFD and asset table, created the screenshots directory structure, and created issues for R1, R4, and R7.
- **Saim Ali Shahid (020) – Threat Analyst:** Verified the STRIDE threats, uploaded the final lab report PDF, and created issues for R6 and R12.

8. Answers to Practice Questions

1. List 10 assets and rate them:
(See Section 3 Asset Table above. For example: A1 User Passwords (C, I - High), A2 Digital Resources (C, I - High), A7 System Uptime (A - High), A3 Resource Metadata (I - Medium), etc.)

2. Threat for each STRIDE letter:

- **S (Spoofing):** Stolen session token used to upload fake resources (Librarian).
- **T (Tampering):** Malicious file altered in transit (Resource data flow).
- **R (Repudiation):** Librarian denies deleting a resource (Process 3.0).
- **I (Info Disclosure):** Unencrypted database backup exposed (D2).
- **D (Denial of Service):** Botnet floods the search function (Process 2.0).
- **E (Elevation of Privilege):** Student manipulates account changes to gain Admin rights (Process 4.0).

3. A student changes resource_id=101 **to** 102 **to see a restricted file:**

- **STRIDE Type:** Elevation of Privilege (or Information Disclosure).
- **Vulnerability:** Broken Access Control (Insecure Direct Object Reference).
- **OWASP Category:** A01:2021-Broken Access Control.
- **Likelihood & Impact:** L: 4, I: 4. **Score:** 16 (High).
- **Two Controls:** 1) Implement server-side role checks to verify the user has access to the specific file. 2) Use UUIDs (random strings) instead of sequential IDs in the URL.

4. The payment gateway sends fee confirmations to P5 without a signature (Adapted: The Librarian uploads a resource without a digital signature):

- **Threat:** Tampering. An attacker could alter the file in transit.
- **Trust Boundary Crossed:** TB1 (Internet to Application Server).
- **Mitigation:** Implement cryptographic hashing (e.g., SHA-256) and digital signatures to verify file integrity upon receipt.

5. On result day 5,000 students open the portal at once (Adapted: On exam week, 5,000 students search for a paper):

- **Security Objective Hit:** Availability.
- **Design-phase controls:** Load balancing architecture to distribute traffic; efficient database indexing to speed up search queries.
- **Deployment-phase controls:** Auto-scaling cloud infrastructure (increase servers automatically); Web Application Firewall (WAF) rate limiting.

6. The admin account has the same password as its username on the test server, and the test server is copied to production:

- **SDLC Phase Failed:** Deployment (or Maintenance).
- **Activity that would have caught it:** Secure Configuration Hardening (or a pre-deployment security audit/penetration test).

7. A teacher says they never changed a mark but the Grades DB shows a change:

- **STRIDE Type:** Repudiation.
- **Asset that must exist:** Audit Log (D3). It must be immutable and capture the User ID, timestamp, and the exact change made to solve the dispute.

8. A new feature, a mobile app, is planned for next year:

- I would repeat the **Design** phase (to draw a new DFD for the mobile app architecture and data flows) and the **Threat Modelling** activity (STRIDE) to identify new threats. I would also repeat the **Testing** phase to ensure the new mobile API is secure before deployment.

9. Conclusion

In this lab, we successfully created a threat model for a University Digital Library. We learned how to map data flows, identify trust boundaries, and use the STRIDE methodology to find vulnerabilities. The hardest part was accurately calculating the Risk Score (Likelihood × Impact) and ensuring every threat had a specific, actionable mitigation. Using GitHub Issues helped our group track which threats needed immediate attention and ensured everyone contributed to the repository. This lab highlighted that security must be built into the Design phase, as fixing flaws after release is much more expensive.

10. Role Sign-Off

- **Group Lead / Facilitator:** M Hammad (051) - *Signature: M. Hammad*
- **Asset and Boundary Analyst:** Ali Hamza (027) - *Signature: A. Hamza*
- **Threat Analyst:** Saim Ali Shahid (020) - *Signature: S.A. Shahid*
- **GitHub Maintainer:** M Hammad (051) - *Signature: M. Hammad*